CSC 402 | Lab 4 Report

1. Introduction and Objectives

This lab explores the Android Activity entry point and launcher configuration, state modeling with Kotlin data classes, stateless Jetpack Compose screens, previews, and event-driven state updates. The in-lab Attendance application and the individual StudyPlanner application were implemented and tested.

2. Task 1 — Activity and Launcher

Q1.1: The activity android:name value is .MainActivity.

Q1.2: The launcher intent filter contains <action android:name="android.intent.action.MAIN" /> and <category android:name="android.intent.category.LAUNCHER" />.

Q1.3: Inside onCreate, the code calls super.onCreate(savedInstanceState), then enableEdgeToEdge(), then setContent { ... }.

Q1.4: After temporarily commenting out the LAUNCHER category, the Lab4 icon was not present in the app drawer.

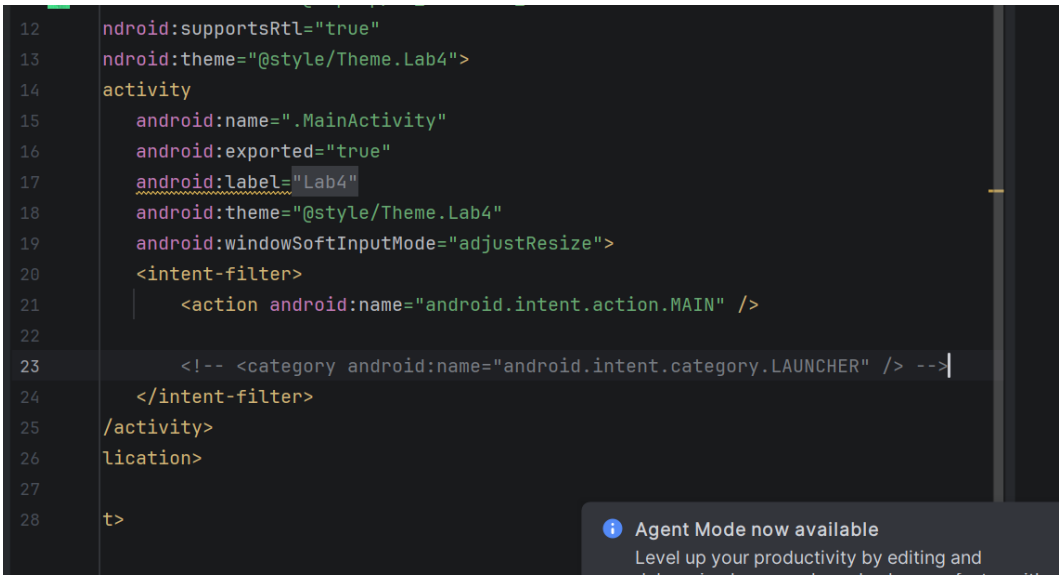


Figure 1. LAUNCHER category temporarily disabled in the manifest.

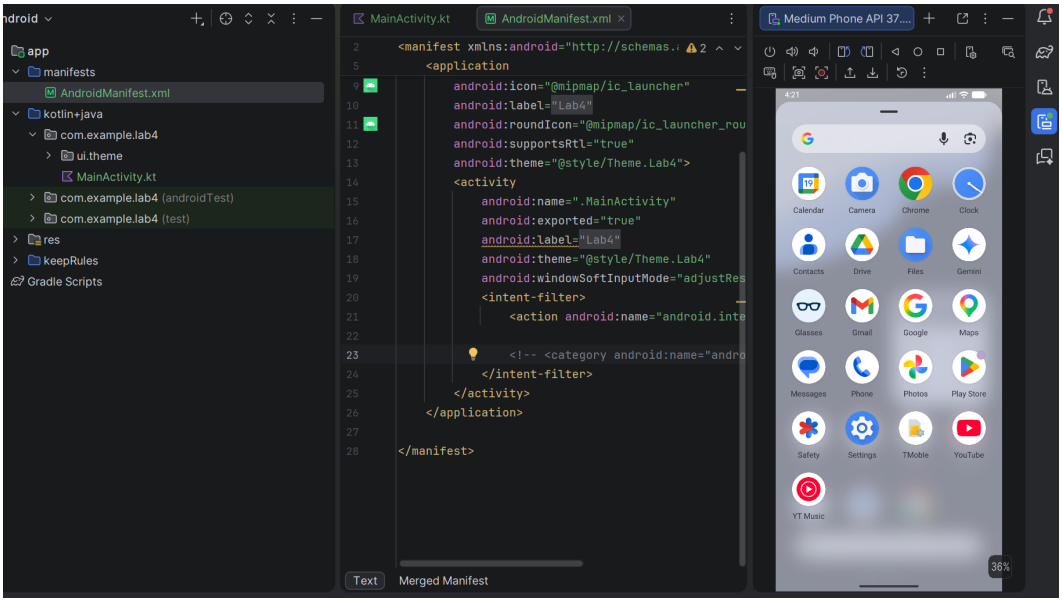


Figure 2. App drawer after disabling the launcher category.

Q1.5: After restoring LAUNCHER, the app could be launched again and displayed Hello Android!.

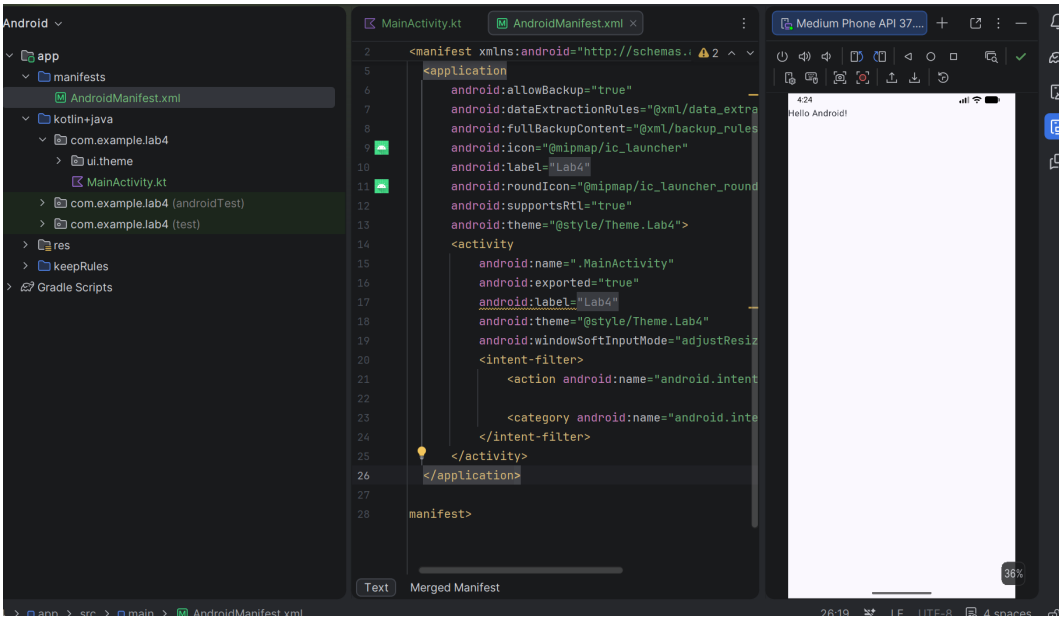


Figure 3. Lab4 launches after the LAUNCHER category is restored.

3. Task 2 — Model the UI State

The Attendance screen displays a session title, a check-in count, and a Check in button. The data class AttendanceUiState contains sessionTitle (String), checkedInCount (Int), and isCheckedIn (Boolean). The user event is onCheckIn.

4. Task 3 — Stateless Composable and Previews

AttendanceScreen accepts uiState and onCheckIn as parameters. It shows the title and count and disables the button when isCheckedIn is true. Two Compose previews represent different UI states: 12 check-ins before the action and 13 after the action.

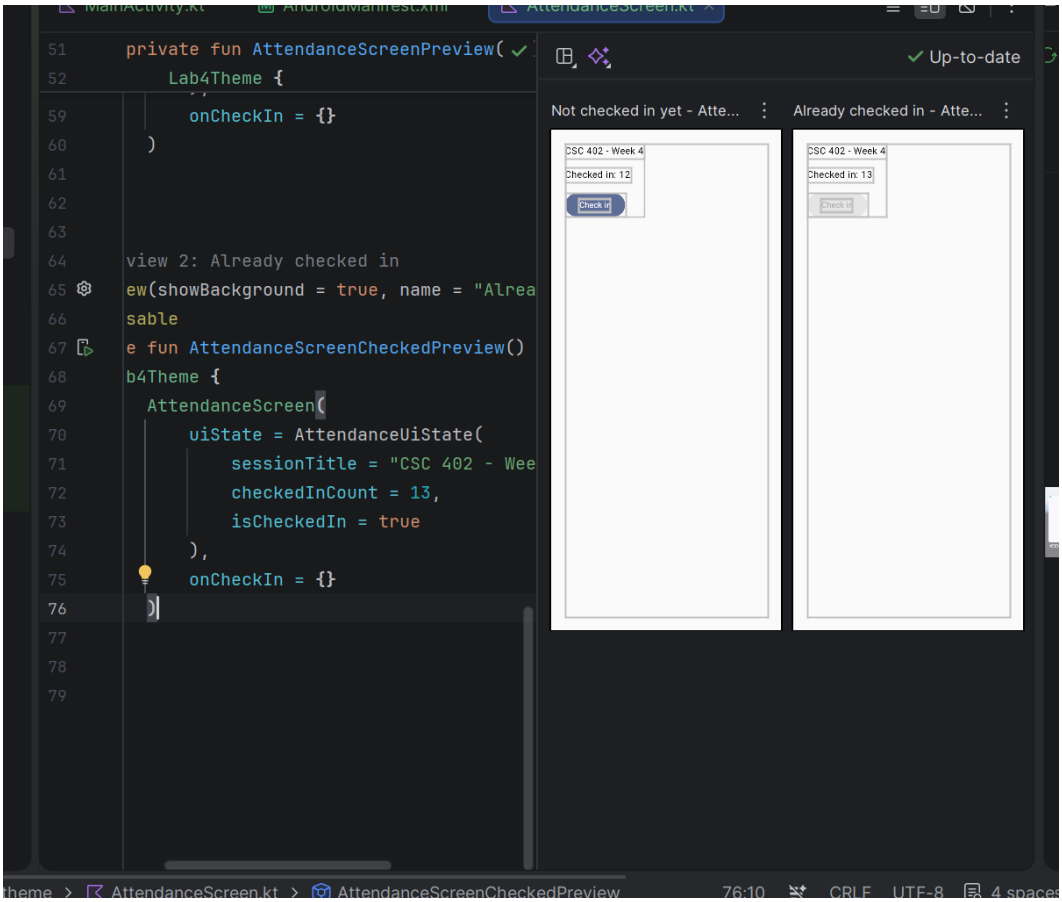


Figure 4. AttendanceScreen previews showing enabled and disabled button states.

5. Task 4 — Route, Events, and State Updates

AttendanceRoute owns a remembered mutable AttendanceUiState. On button press, it creates and assigns a new state using uiState.copy(checkedInCount = uiState.checkedInCount + 1, isCheckedIn = true). Compose recomposes the screen with the updated values.

Observed result: the count changed from 12 to 13 and the Check in button became disabled.

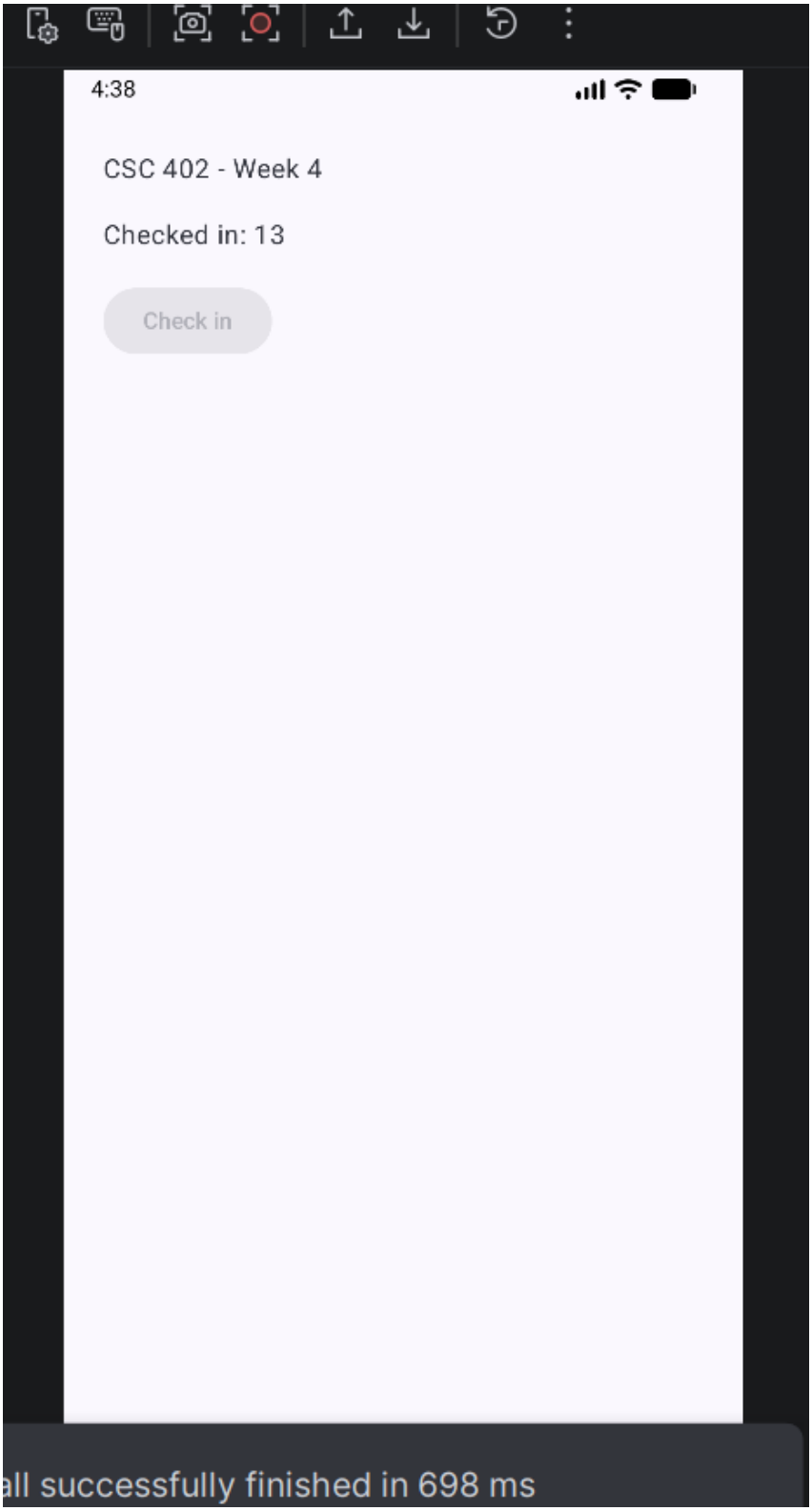


Figure 5. Attendance app after a successful check-in.

Broken update experiment: replacing the callback with uiState.checkedInCount + 1 only computes a value; it does not assign new state. The count therefore remains 12 and the button stays enabled. The correct copy-based handler was restored afterward.

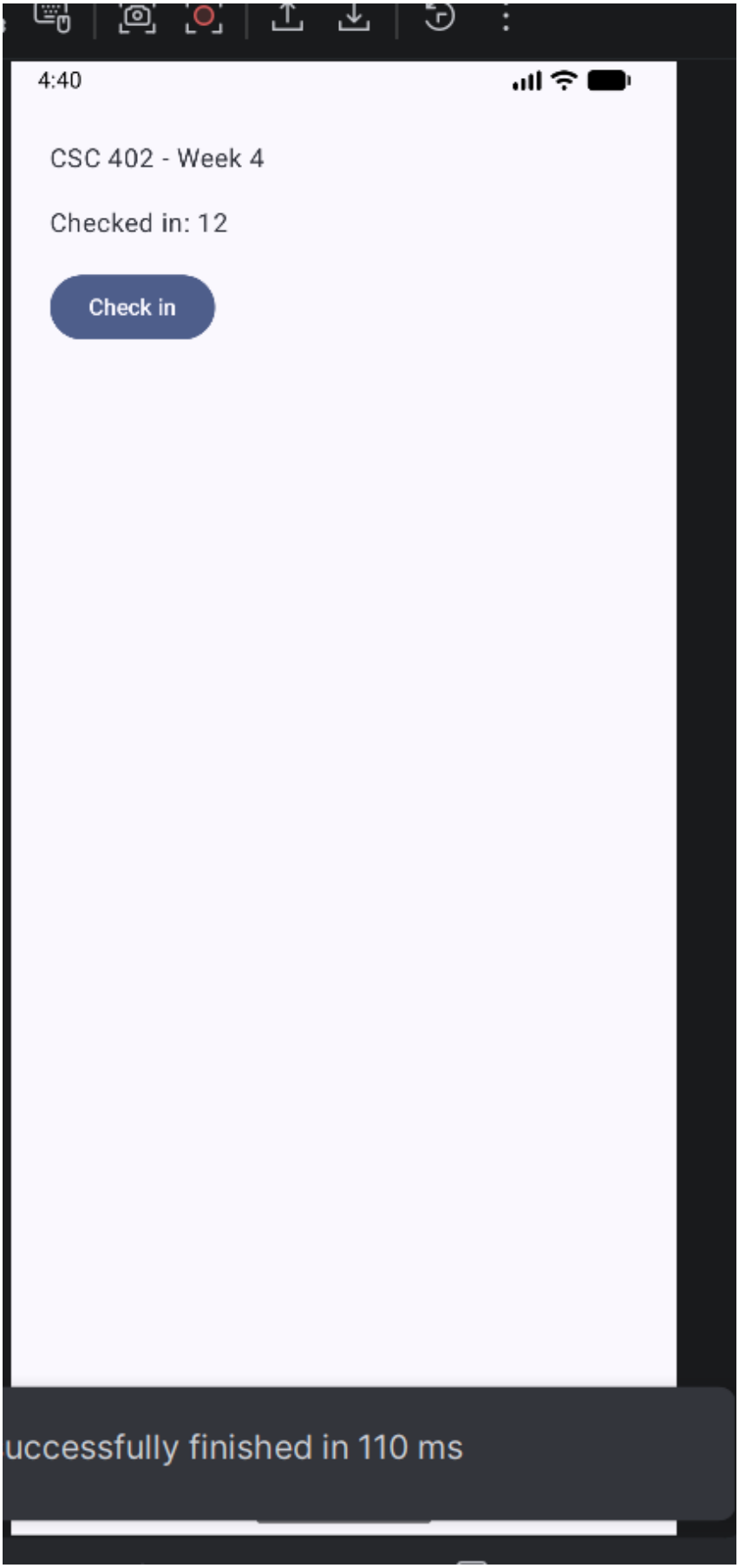

4:40
CSC 402 - Week 4
Checked in: 12
Check in
uccessfully finished in 110 ms

Figure 6. Broken update experiment; the state does not change.

6. Individual Assignment — StudyPlanner (5 Marks)

6.1 Paper Sketch and Screen Plan (P1.1)

The planned home screen contains a greeting, study progress, and a Complete a Task button. The changing state is completedTasks; totalTasks and greeting are provided in the UI state. The user action completes one additional task.

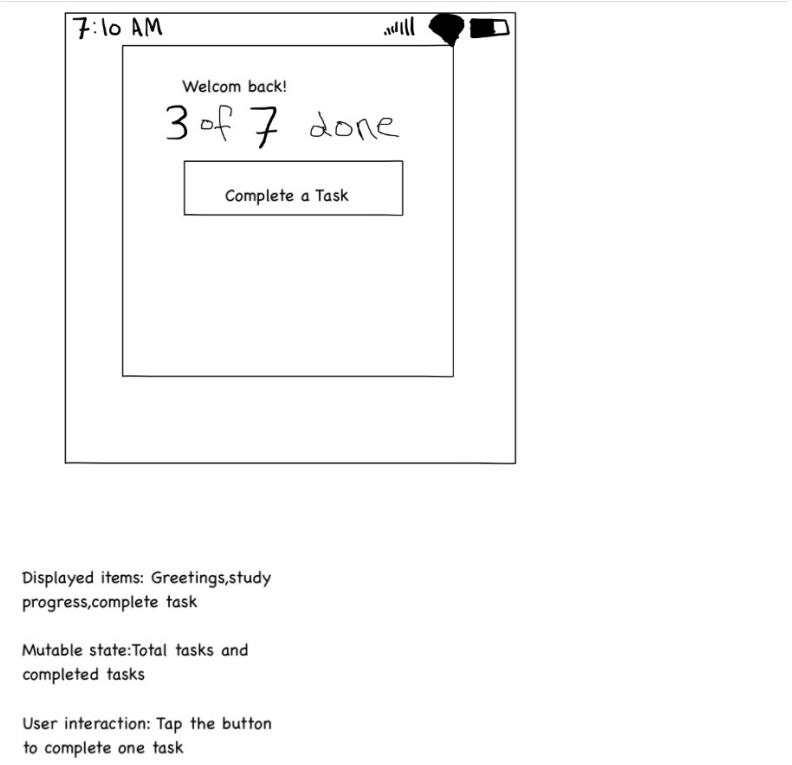


Figure 7. Hand-drawn StudyPlanner screen and notes.

6.2 Project and UI State (P1.2–P1.3)

A separate Android Studio project named StudyPlanner was created. PlannerUiState is a Kotlin data class with default values: greeting = "Welcome back!", totalTasks = 7, and completedTasks = 3. The implemented package is com.example.studyplanner; the lab handout specifies sa.edu.iau.cshj.studyplanner, so this package-name requirement should be checked before submission.

6.3 Stateless Home Screen and Previews (P1.4–P1.5)

PlannerHomeScreen(uiState, onCompleteTask, modifier) displays the greeting, progress in the format “3 of 7 done”, and a Complete a Task button. The button is enabled only while completedTasks < totalTasks. Two previews show an in-progress state (3/7) and a completed state (7/7).

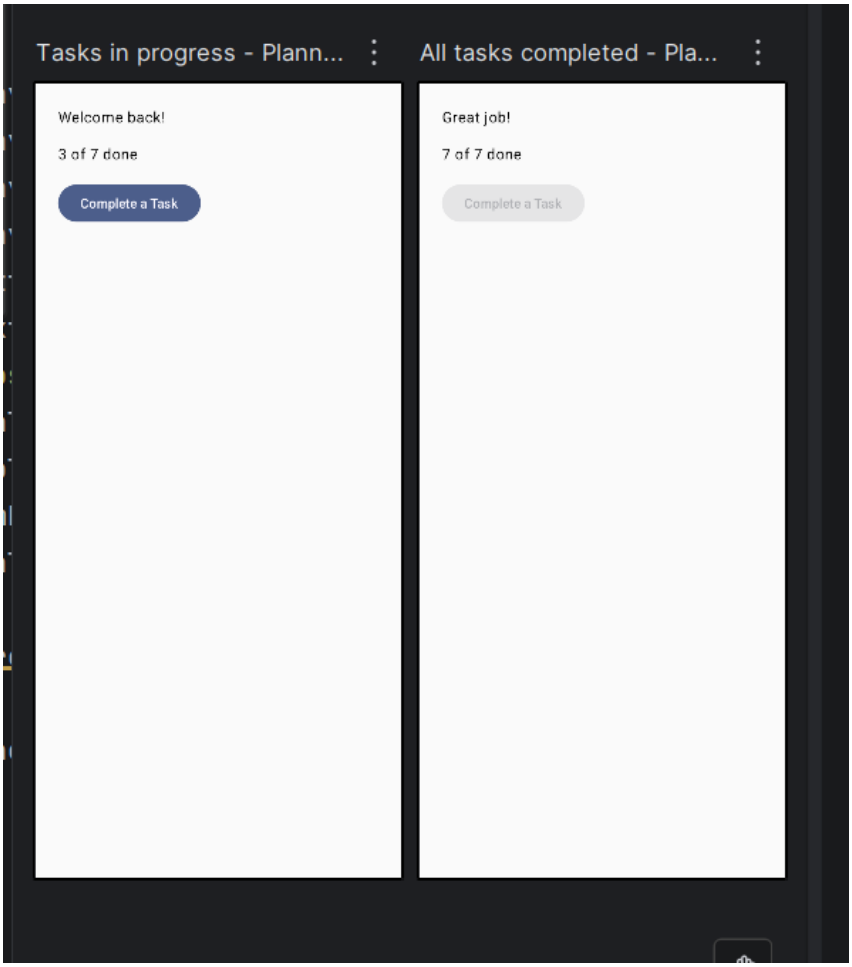


Figure 8. StudyPlanner previews: tasks in progress and all tasks completed.

6.4 Route, Interaction, and Testing (P1.6)

PlannerHomeRoute owns the mutable state using remember and mutableStateOf. Its onCompleteTask handler assigns uiState = uiState.copy(completedTasks = uiState.completedTasks + 1). MainActivity calls PlannerHomeRoute inside the StudyPlannerTheme. The running application progressed from 3/7 to 4/7, then 5/7, 6/7, and 7/7. At 7/7, the button was disabled.

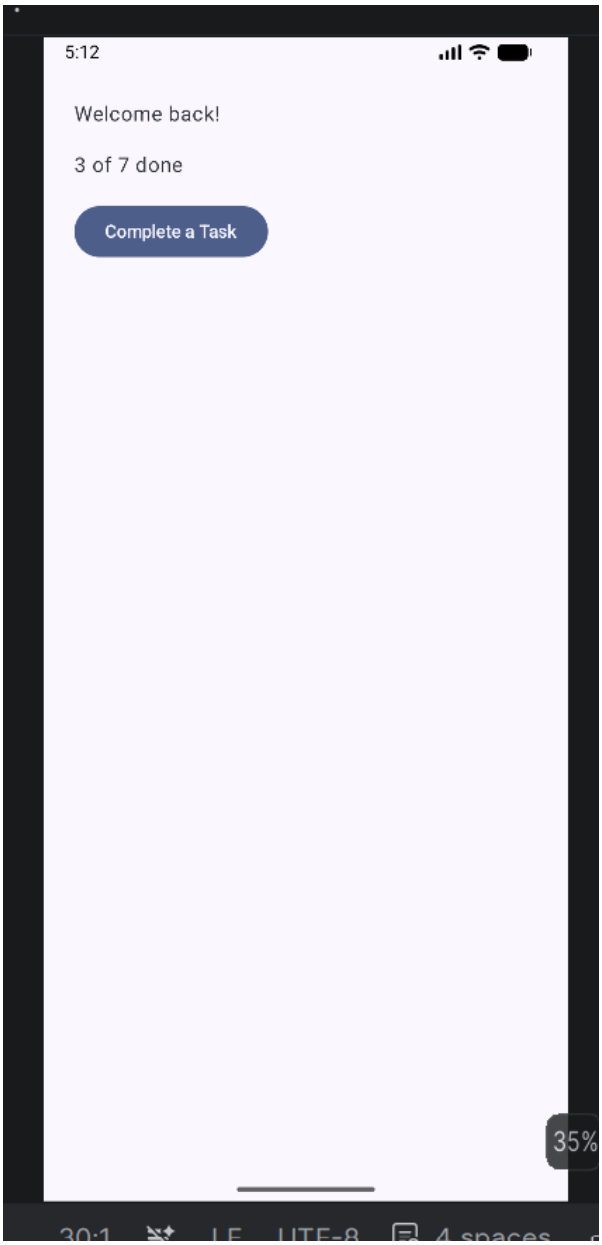


Figure 9. StudyPlanner running on the emulator during task completion.

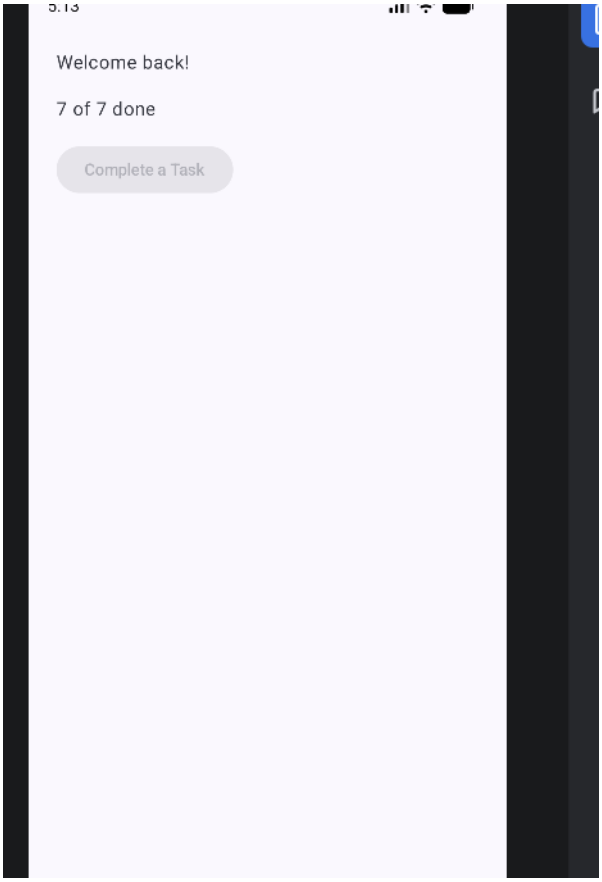


Figure 10. Additional StudyPlanner running-state evidence.

6.5 Manifest Configuration (P1.7)

AndroidManifest.xml declares exactly one Activity, .MainActivity, with android:exported="true". Its intent filter contains android.intent.action.MAIN and android.intent.category.LAUNCHER. The application label uses @string/app_name. The configured label should be checked in strings.xml.

7. Results and Conclusion

Both Compose projects were run and tested. Lab4 demonstrated that launcher configuration controls discoverability and that recomposition depends on assigning updated state. StudyPlanner demonstrated a stateless home screen, two meaningful previews, a Route that updates state with copy(), and a completion button that becomes disabled at the task limit.

8. AI Assistance Disclosure

AI was used when there was a little bit of confusion with the code error and the running issues.